CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

# Objective 9: Apply Prompting Techniques

**Exam objective:** Apply prompting techniques—zero-shot, few-shot, chain-of-thought/extended reasoning, example selection, explicit instructions, and structured output—choosing the right technique and knowing when examples outperform additional instructions.

---

**Q1** **A bank wants Claude to classify loan-application notes into one of six risk categories using highly specific, idiosyncratic internal terminology that is not well represented in general training data. What technique is MOST likely to improve accuracy quickly?**

A. Zero-shot prompting with only a category list

**B. Few-shot prompting with 4-6 representative labeled examples that illustrate the idiosyncratic terminology and edge cases**

C. Chain-of-thought reasoning with no examples at all

D. Simply increasing the system prompt's politeness level

**CORRECT ANSWER** B

**WHY** When a task relies on domain-specific or idiosyncratic patterns that are hard to describe abstractly, well-chosen few-shot examples teach the mapping more reliably than instructions alone. Zero-shot lacks grounding in the specific terminology, chain-of-thought without examples doesn't address the terminology gap, and tone adjustments are irrelevant to classification accuracy.

**Q2** An architect is deciding whether to add more explicit written instructions or add examples to improve a healthcare intake-summarization prompt that keeps omitting a required "allergies" section. The rule is simple and easy to state precisely. What is the BEST first fix?

A. **Add a clear, explicit instruction: "Always include an 'Allergies' section, even if the answer is 'None reported.'"**

B. Add ten few-shot examples before trying an explicit instruction

C. Switch to chain-of-thought reasoning to help the model "think harder" about allergies

D. Remove the requirement since it is hard to enforce

**CORRECT ANSWER** A

**WHY** When a requirement is a simple, explicit, easily stated rule, a direct instruction is typically more efficient and reliable than examples or added reasoning steps. Jumping straight to many examples is costlier than needed for a simple rule, chain-of-thought doesn't fix an omission caused by unclear instructions, and removing the requirement ignores the actual business need.

**Q3** **A manufacturing QA system needs Claude to decide, with justification, whether a defect report indicates a critical safety issue requiring immediate line shutdown—a judgment call involving multiple weighed factors. Which technique is MOST appropriate for improving the quality of this judgment?**

A Zero-shot prompting with just the defect report and no guidance

B **Chain-of-thought / extended reasoning prompting that asks the model to explicitly weigh relevant factors (severity, frequency, affected systems) before concluding**

C Providing one single rigid example to copy verbatim

D Asking the model to answer in exactly three words

**CORRECT ANSWER** B

**WHY** Multi-factor judgment calls benefit from explicit reasoning steps (chain-of-thought) that surface the weighing of relevant factors before a conclusion, improving both accuracy and auditability. Zero-shot with no guidance under-specifies the task, one rigid example risks overfitting to a single pattern, and forcing a terse answer discards the reasoning needed for high-stakes judgment.

**Q4** **A retail company's prompt for generating product taglines keeps producing taglines that don't match the brand's quirky tone, despite a long written description of the tone in the system prompt. What is the MOST effective next step?**

A Make the written tone description even longer and more abstract

**B Add 3-5 concrete few-shot examples of taglines that exemplify the desired tone, since tone/style is often easier to demonstrate than to describe**

C Remove the tone requirement from the task

D Switch to chain-of-thought prompting, since tone is a reasoning problem

**CORRECT ANSWER** B

**WHY** Style and tone are classic cases where concrete examples communicate the target far more effectively than abstract descriptive instructions, since "quirky" is subjective and hard to define precisely in words. Lengthening abstract description tends to have diminishing returns, dropping the requirement ignores the business goal, and tone matching is not fundamentally a multi-step reasoning problem.

**Q5** **An architect selecting few-shot examples for a legal clause-classification prompt has 500 historical labeled clauses to choose from. Which example-selection strategy is MOST likely to maximize generalization?**

A Pick the first 5 examples in the dataset regardless of content

**B Pick a small, diverse set of examples that cover distinct clause types and known edge cases, rather than near-duplicates of one pattern**

C Pick the 5 longest examples, since longer text is inherently more informative

D Pick 5 examples that are all the same clause type to reinforce one pattern strongly

**CORRECT ANSWER** B

**WHY** Diverse, representative examples that cover the range of categories and edge cases generalize better than redundant or arbitrarily selected ones. Picking arbitrarily, by length, or all from one category risks biasing the model toward a narrow pattern and failing to generalize to case diversity.

**Q6** **A government call-center agent must answer FAQs using only approved language from a style guide, with zero tolerance for paraphrasing beyond what's permitted. Which prompting approach is MOST appropriate?**

A Zero-shot prompting with vague tone guidance only

B **Explicit, highly specific instructions (e.g., quote exact approved phrasing, flag anything not in the style guide) combined with structured output constraints, rather than relying on inferred style from loose examples**

C Chain-of-thought reasoning focused on creative rephrasing

D Few-shot examples that intentionally include varied, inconsistent phrasing

**CORRECT ANSWER** B

**WHY** When compliance requires near-literal adherence to approved language, explicit and unambiguous instructions plus structured constraints are more reliable than relying on inferred patterns. Vague guidance underconstrains the task, encouraging creative rephrasing works against the compliance goal, and inconsistent examples would teach the wrong lesson.

**Q7** A SaaS analytics company wants Claude to generate SQL queries from natural-language questions against a known schema. Initial zero-shot results are inconsistent in table/column naming. What is the BEST combination of techniques to try next?

**A** Provide the schema explicitly plus 3-4 few-shot examples of question-to-SQL pairs using that exact schema

B Remove the schema from the prompt to reduce token usage

C Ask the model to guess column names based on general SQL conventions

D Switch exclusively to chain-of-thought with no schema or examples

CORRECT ANSWER **A**

WHY Grounding the model with the actual schema plus concrete worked examples directly addresses the naming-consistency problem by anchoring outputs to the real structure. Removing the schema would make naming worse, guessing conventions ignores the actual schema, and reasoning steps alone don't supply the missing schema information.

**Q8** **An architect is prompting Claude to extract structured fields (name, date, amount, account number) from scanned invoice OCR text for a logistics company. Which instruction technique BEST reduces downstream parsing errors?**

A Ask for "a summary of the invoice" in free-form prose

B **Explicitly specify the exact output schema/field names and require JSON-only output with defined handling for missing fields (e.g., null)**

C Ask the model to describe the invoice in a poem for variety

D Leave the output format unspecified so the model can choose what's best each time

CORRECT ANSWER **B**

WHY Precise schema specification plus explicit rules for missing-field handling is the standard technique for minimizing downstream parsing errors in structured extraction tasks. Free-form prose, creative reformatting, and unspecified format all introduce inconsistency that breaks programmatic parsing.

**Q9** **A customer-support team iterating on a refund-eligibility prompt finds that after several rounds of adding more and more explicit rules, performance has plateaued and new edge cases still fail. What is the MOST productive next step in iterative refinement?**

A Keep adding more explicit rules indefinitely

**B Step back and add a small set of carefully chosen few-shot examples covering the failing edge cases, since instructions may have reached diminishing returns for this type of nuanced judgment**

C Abandon the refund-eligibility feature entirely

D Reduce the prompt to a single sentence to "simplify" it

**CORRECT ANSWER** B

**WHY** When instruction-stacking plateaus, especially for nuanced edge-case judgment, targeted examples addressing the specific failures are often more effective than further rule accumulation. Endless rule addition has diminishing returns, abandoning the feature is an overreaction, and drastically simplifying the prompt would likely worsen performance.

**Q10** **A manufacturing predictive-maintenance assistant must explain its reasoning to engineers when recommending equipment shutdown, so humans can audit the logic. Which prompting technique BEST supports this requirement?**

A Zero-shot prompting that asks only for a final yes/no answer

**B Extended reasoning / chain-of-thought prompting that asks the model to show the intermediate factors considered before the final recommendation**

C Few-shot examples with no final answer shown, only questions

D Structured output requiring a single numeric code with no explanation

CORRECT ANSWER **B**

WHY When human auditability of reasoning is a requirement, chain-of-thought/extended reasoning that surfaces intermediate considerations directly serves that goal. A terse yes/no or numeric code provides no auditable reasoning trail, and examples without shown answers don't address the explanation requirement.

**Q11** **An architect wants to reduce hallucinated citations in a legal research assistant's output. Which prompting change is MOST likely to help, independent of any retrieval system changes?**

**A Explicitly instruct the model to only cite sources present in the provided context and to state "no supporting source found" rather than inventing one, and consider few-shot examples demonstrating this refusal behavior**

B Ask the model to cite more sources to increase credibility

C Remove all instructions about citations so the model behaves naturally

D Instruct the model to always sound confident regardless of source availability

CORRECT ANSWER **A**

WHY Explicit grounding instructions plus examples demonstrating the "no source found" fallback directly target hallucination by giving the model a safe, instructed alternative to inventing citations. Asking for more citations increases hallucination risk, removing instructions removes the safeguard, and demanding confidence actively encourages fabrication.

**Q12** **A telecom billing dispute assistant performs best on simple disputes with zero-shot prompting but struggles on multi-step disputes involving several billing cycles and partial credits. What is the BEST targeted fix for the complex cases specifically?**

A. Apply the same zero-shot approach uniformly to all disputes regardless of complexity

**B. For complex, multi-cycle disputes, introduce chain-of-thought prompting that walks through each billing cycle sequentially before producing a final resolution**

C. Shorten the prompt for complex disputes to save tokens

D. Disable the assistant for any dispute involving more than one billing cycle

**CORRECT ANSWER** B

**WHY** Complex, multi-step disputes benefit from explicit sequential reasoning that mirrors the structure of the problem (cycle by cycle), improving accuracy on compound cases. Applying zero-shot uniformly ignores the complexity difference, shortening the prompt would likely worsen performance, and disabling the assistant abandons the use case rather than solving it.

**Q13** **A product team is prompting Claude to write release notes from raw commit logs. They notice that providing 2 highly polished example release notes produces much better results than a 3-paragraph description of "what makes good release notes." What principle does this illustrate?**

A Examples are always strictly better than instructions in every scenario

B **For tasks where the desired output is primarily a matter of style, structure, and voice that is easier to demonstrate than describe, well-chosen examples can outperform purely descriptive instructions**

C Commit logs are irrelevant to the quality of release notes

D Instructions should never be used alongside examples

CORRECT ANSWER **B**

WHY This scenario is a textbook case of style/structure being easier to show than tell—examples transmit tacit conventions (format, voice, level of detail) that are hard to fully specify in prose. It does not mean examples always beat instructions universally, commit logs remain the substantive input, and examples and instructions are commonly combined rather than mutually exclusive.

**Q14** **An insurance underwriting assistant must output a risk score (1-100) plus a short rationale, in a specific JSON structure, for each application. Which technique set is MOST appropriate to combine?**

A Chain-of-thought only, with no structured output specification

**B** **Explicit structured-output instructions defining the JSON schema, combined with 2-3 few-shot examples showing the exact expected format and a realistic rationale**

C Zero-shot prompting with no schema and no examples

D Only few-shot examples, with the schema deliberately left unspecified "for flexibility"

**CORRECT ANSWER** B

**WHY** Combining explicit schema instructions with concrete format examples reinforces both what the structure must be and what a good rationale looks like, which is the most reliable combination for structured, judgment-involving output. CoT alone doesn't guarantee format compliance, zero-shot with no schema invites inconsistency, and intentionally vague schema undermines the structured-output goal.

**Q15** A logistics company's prompt engineer is deciding whether to invest time curating five excellent few-shot examples or writing twenty additional instruction bullet points for a shipment-exception classification task. Which consideration should MOST influence this decision?

A Always choose more instructions, since instructions are cheaper to write

B **Assess whether the failure mode is about the model not understanding abstract category boundaries (favoring examples that demonstrate the boundary) versus not knowing an explicit, statable rule (favoring direct instructions)**

C Flip a coin, since both techniques are interchangeable

D Choose few-shot examples unconditionally, since examples are always superior for classification tasks

CORRECT ANSWER **B**

WHY The right choice depends on the nature of the failure: ambiguous category boundaries are often best taught by demonstration, while clearly statable rules are best handled by direct instruction. Defaulting to one technique regardless of failure mode, or treating the choice as arbitrary, ignores the diagnostic step that should drive technique selection.

**Q16** A team uses Claude's structured outputs feature (constrained decoding against a JSON schema) for a ticket-classification feature with an enum field defined as `["Open", "Closed", "Pending"]`. A teammate proposes also adding `"open"` (lowercase) as a distinct fourth enum value to capture a separate internal state. What should the architect flag as the key risk?

A Structured outputs cannot use enums at all, so the schema is invalid

**B String enum and const values are not guaranteed to match the schema's exact capitalization even though the response validates successfully, so defining two enum values that differ only in capitalization (e.g., "Open" vs "open") creates values the application cannot reliably distinguish; enum comparisons should be done case-insensitively**

C Adding a fourth enum value will cause every request to return a 400 error

D Enum values are case-sensitive and always returned in exactly the casing defined, so this change is completely safe

**CORRECT ANSWER** B

**WHY** Structured outputs constrain the shape of a response via constrained decoding, but documented behavior notes that string enum/const casing is not guaranteed, with no error signal when it mismatches—so near-duplicate enum values differing only in case are indistinguishable downstream, and comparisons should be case-insensitive. Enums are supported, this change would not cause a 400, and casing is not guaranteed to be exact.

**Q17** **An architect reads older documentation recommending that assistant-turn prefilling be used to skip Claude's conversational preamble and force a consistent output structure. The team is deploying on a current-generation model family where such prefilled requests return a 400 error on the last assistant turn. What is the BEST way to achieve the same consistent-structure goal on this model?**

A Keep attempting the prefill technique repeatedly until it succeeds, since documentation recommends it

**B** **Rely on explicit structured-output/schema instructions (and, where available, a structured-output/constrained-decoding feature) plus clear output-format instructions in the system prompt, since prefilling the final assistant turn is no longer supported on this model family and the API-enforced constraint takes precedence over the older technique**

C Switch to a much older model solely to preserve the prefill technique, regardless of other requirements

D Abandon any attempt at consistent output structure, since no alternative exists

CORRECT ANSWER **B**

WHY When documented guidance conflicts with an enforced API constraint, the constraint wins; since prefilling the last assistant turn is rejected on this model family, the consistent-structure goal should be met with explicit schema/output-format instructions (and structured-output features where supported) instead. Repeating a rejected technique wastes effort, downgrading models solely to preserve a deprecated trick is not a sound architectural trade-off, and alternatives for achieving structure do exist. --